

1 React Introduction

Definition:

React is a JavaScript library used to build **user interfaces (UI)**, especially **single-page applications (SPA)**.

Key Features:

- Component-based architecture
- Virtual DOM (fast updates)
- Reusable code
- One-way data binding

Example:

```
import React from "react";

function App() {
  return <h1>Hello Mowa!</h1>;
}

export default App;
```

Explanation:

This creates a simple UI showing "Hello Mowa!" using a React component.

2 Arrow Functions in React

Definition:

Arrow functions are a shorter way to write functions in JavaScript, commonly used in React.

Syntax:

```
const functionName = () => { }
```

Example:

```
const Welcome = () => {  
  return <h2>Welcome to React</h2>;  
};
```

With Props:

```
const Hello = (props) => <h1>Hello {props.name}</h1>;
```

Advantages:

- Short syntax
 - No need for `this` binding
 - Cleaner code
-

3 Components

Definition:

Components are **independent, reusable pieces of UI**.

Types:

- **Functional components**
- **Class components**

Example:

```
function Greeting() {  
  return <h1>Hello User</h1>;  
}
```

Usage:

```
<Greeting />
```

Explanation:

Component acts like a function that returns UI.

4 Props (Properties)

Definition:

Props are used to **pass data from parent to child component**.

Example:

```
function Welcome(props) {  
  
  return <h1>Hello {props.name}</h1>;  
  
}
```

```
function App() {  
  
  return <Welcome name="Mowa" />;  
  
}
```

Output:

Hello Mowa

Key Points:

- Read-only
 - Used for communication between components
-

5 State

Definition:

State is used to **store dynamic data** in a component.

Example:

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <>

    <p>{count}</p>

    <button onClick={() => setCount(count + 1)}>Increase</button>

    </>

  );

}
```

Explanation:

- `count` → current value
 - `setCount` → updates value
 - UI updates automatically
-

6 Error Handling in React

Definition:

Error handling is used to **catch and manage errors** in components.

Method: Error Boundary (Class Component)

Example:

```
class ErrorBoundary extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { hasError: false };  
  }  
  
  static getDerivedStateFromError(error) {  
    return { hasError: true };  
  }  
  
  render() {  
    if (this.state.hasError) {  
      return <h1>Something went wrong</h1>;  
    }  
    return this.props.children;  
  }  
}
```

Usage:

```
<ErrorBoundary>  
  <MyComponent />  
</ErrorBoundary>
```

React Router

Definition:

Used to **navigate between pages** without reloading.

React Router

Example:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

```
function Home() {
```

```
  return <h1>Home Page</h1>;
```

```
}
```

```
function About() {
```

```
  return <h1>About Page</h1>;
```

```
}
```

```
function App() {
```

```
  return (
```

```
    <BrowserRouter>
```

```
      <Routes>
```

```
        <Route path="/" element={<Home />} />
```

```
        <Route path="/about" element={<About />} />
```

```
      </Routes>
```

```
    </BrowserRouter>
```

```
  );
```

```
}
```

8 Hooks

Definition:

Hooks allow you to use **state and lifecycle features in functional components**.

Common Hooks:

- `useState`
 - `useEffect`
-

useState Example:

```
const [name, setName] = useState("Mowa");
```

useEffect Example:

```
import { useEffect } from "react";
```

```
useEffect(() => {
```

```
  console.log("Component Mounted");
```

```
}, []);
```

Explanation:

- Runs when component loads
- Used for API calls, side effects
-